

Experiment 2

Name: Sneha Gopal Bhalekar Roll_no : CS05

Aim: To implement a Simple Linear Regression model using Python and predict MPG from Horsepower using OLS.

Objectives:

- Understand SLR
- Load/preprocess dataset
- Perform EDA
- Compute OLS coefficients
- Evaluate with SSE, MSE, R^2
- Visualize regression and residual plots

Software Requirements:

- Python 3.x
- Jupyter/Colab/VS Code
- Libraries: pandas, numpy, matplotlib, seaborn

Theory:

Linear regression models the relationship between a target continuous variable Y and an input feature vector X using a linear equation:

$$\hat{y} = w_0 + w_1 x_1$$

In matrix form, including the bias/intercept term $X_0=1$:

$$\hat{y} = Xw$$

To find the optimal weight vector $\hat{w}$, the Ordinary Least Squares (OLS) method minimizes the Sum of Squared Errors (SSE) between actual target values y and predicted values $\hat{y}$:

$$SSE = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = (y - Xw)^T (y - Xw)$$

Solving for w analytically yields the normal equation:

$$\hat{w} = (X^T X)^{-1} X^T y$$

Note: The inverse $X^T X^{-1}$ exists only if the matrix $X^T X$ is non-singular (i.e., its determinant is non-zero).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving auto-mpg.csv to auto-mpg.csv

```
df = pd.read_csv("auto-mpg.csv")
```

```
df.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin
0	18.0	8	307.0	130	3504	12.0	70	
1	15.0	8	350.0	165	3693	11.5	70	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg             398 non-null   float64
1   cylinders       398 non-null   int64
2   displacement    398 non-null   float64
3   horsepower      398 non-null   object
4   weight          398 non-null   int64
```

```

5   acceleration  398 non-null    float64
6   model year    398 non-null    int64
7   origin        398 non-null    int64
8   car name      398 non-null    object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB

```

```
df.isnull().sum()
```

```

      0
mpg    0
cylinders  0
displacement  0
horsepower  0
weight    0
acceleration  0
model year  0
origin    0
car name  0

```

```
dtype: int64
```

```
df = df.dropna()
```

```

# Select independent variable (X) and dependent variable (Y)
X = df["horsepower"]
Y = df["mpg"]

print(X.head())
print(Y.head())

```

```

0    130
1    165
2    150
3    150
4    140
Name: horsepower, dtype: object
0    18.0
1    15.0
2    18.0
3    16.0
4    17.0
Name: mpg, dtype: float64

```

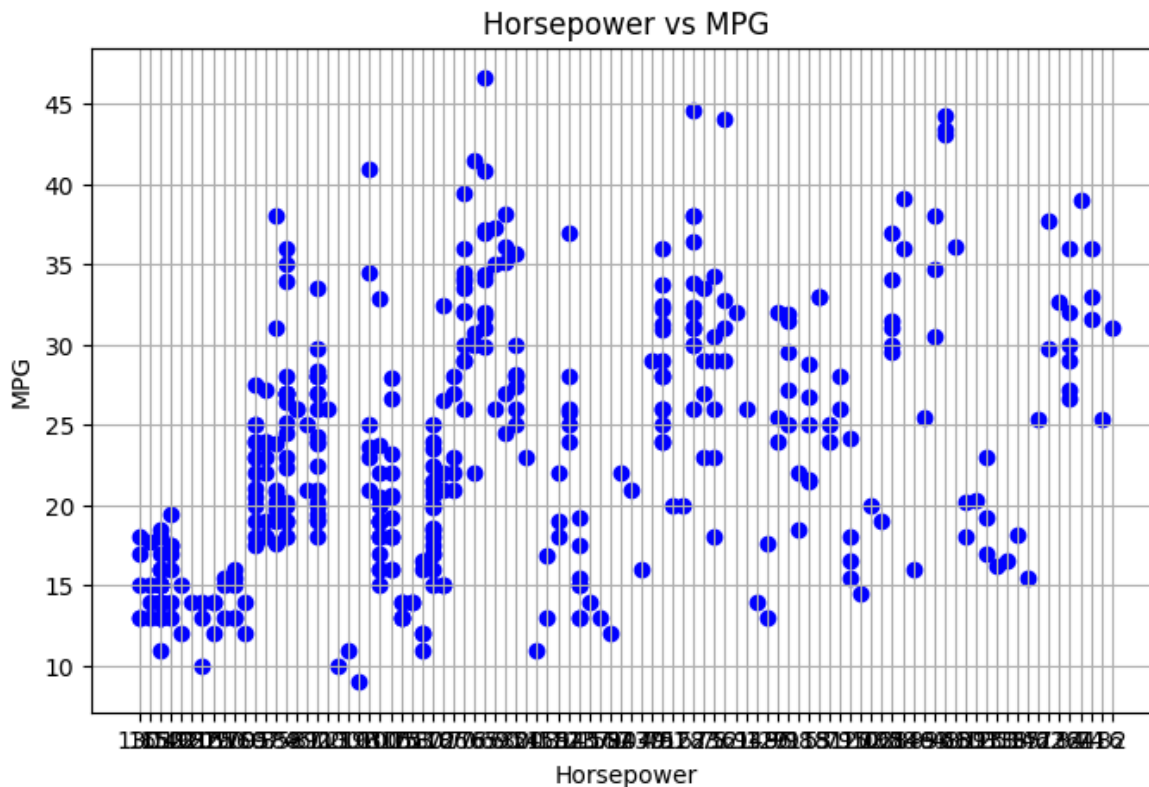
```
import matplotlib.pyplot as plt
```

```

plt.figure(figsize=(8,5))
plt.scatter(X, Y, color="blue")

```

```
plt.xlabel("Horsepower")
plt.ylabel("MPG")
plt.title("Horsepower vs MPG")
plt.grid(True)
plt.show()
```



```
import numpy as np

# Convert X into a 2D array
X = X.values.reshape(-1,1)

# Add bias (column of ones)
X_bias = np.c_[np.ones((X.shape[0],1)), X]

print(X_bias[:5])
```

```
[[1.0 '130']
 [1.0 '165']
 [1.0 '150']
 [1.0 '150']
 [1.0 '140']]
```

```
print(df["horsepower"].dtype)
```

```
object
```

```
import numpy as np
```

```
df["horsepower"] = df["horsepower"].replace("?", np.nan)
```

```
df["horsepower"] = pd.to_numeric(df["horsepower"], errors="coerce")
```

```
df = df.dropna(subset=["horsepower"])
```

```
print(df["horsepower"].dtype)
print(df["horsepower"].isnull().sum())
```

```
float64
0
```

```
correlation = df[["horsepower", "mpg"]].corr()
print(correlation)
```

```

           horsepower      mpg
horsepower    1.000000 -0.778427
mpg           -0.778427  1.000000
```

```
import numpy as np
import pandas as pd

# Clean the columns
df["horsepower"] = pd.to_numeric(df["horsepower"], errors="coerce")
df["mpg"] = pd.to_numeric(df["mpg"], errors="coerce")

# Remove missing values
df = df.dropna(subset=["horsepower", "mpg"])

# Convert to NumPy arrays
X = df["horsepower"].to_numpy(dtype=float).reshape(-1, 1)
Y = df["mpg"].to_numpy(dtype=float)

# Add bias term
X_bias = np.hstack((np.ones((X.shape[0], 1)), X))

# Calculate OLS weights
w = np.linalg.inv(X_bias.T @ X_bias) @ X_bias.T @ Y

print("Intercept =", w[0])
print("Slope =", w[1])
```

```
Intercept = 39.9358610211705
Slope = -0.15784473335365387
```

```
Y_pred = X_bias @ w
```

```
print(Y_pred[:10])
```

```

[19.41604569 13.89148002 16.25915102 16.25915102 17.83759835  8.68260382
  5.21001968  5.99924335  4.42079602  9.94536168]
```

```
residuals = Y - Y_pred
```

```
print(residuals[:10])
```

```
[-1.41604569  1.10851998  1.74084898 -0.25915102 -0.83759835  6.31739618  
 8.78998032  8.00075665  9.57920398  5.05463832]
```

```
# Sum of Squared Errors
```

```
SSE = np.sum(residuals ** 2)
```

```
# Mean Squared Error
```

```
MSE = SSE / len(Y)
```

```
# R2 Score
```

```
SST = np.sum((Y - np.mean(Y)) ** 2)
```

```
R2 = 1 - (SSE / SST)
```

```
print("SSE =", SSE)
```

```
print("MSE =", MSE)
```

```
print("R2 =", R2)
```

```
SSE = 9385.915871932419
```

```
MSE = 23.943662938603108
```

```
R2 = 0.6059482578894348
```

```
plt.figure(figsize=(8,5))
```

```
plt.scatter(X, Y, color="blue", label="Actual Data")
```

```
plt.plot(X, Y_pred, color="red", linewidth=2, label="Regression Line")
```

```
plt.xlabel("Horsepower")
```

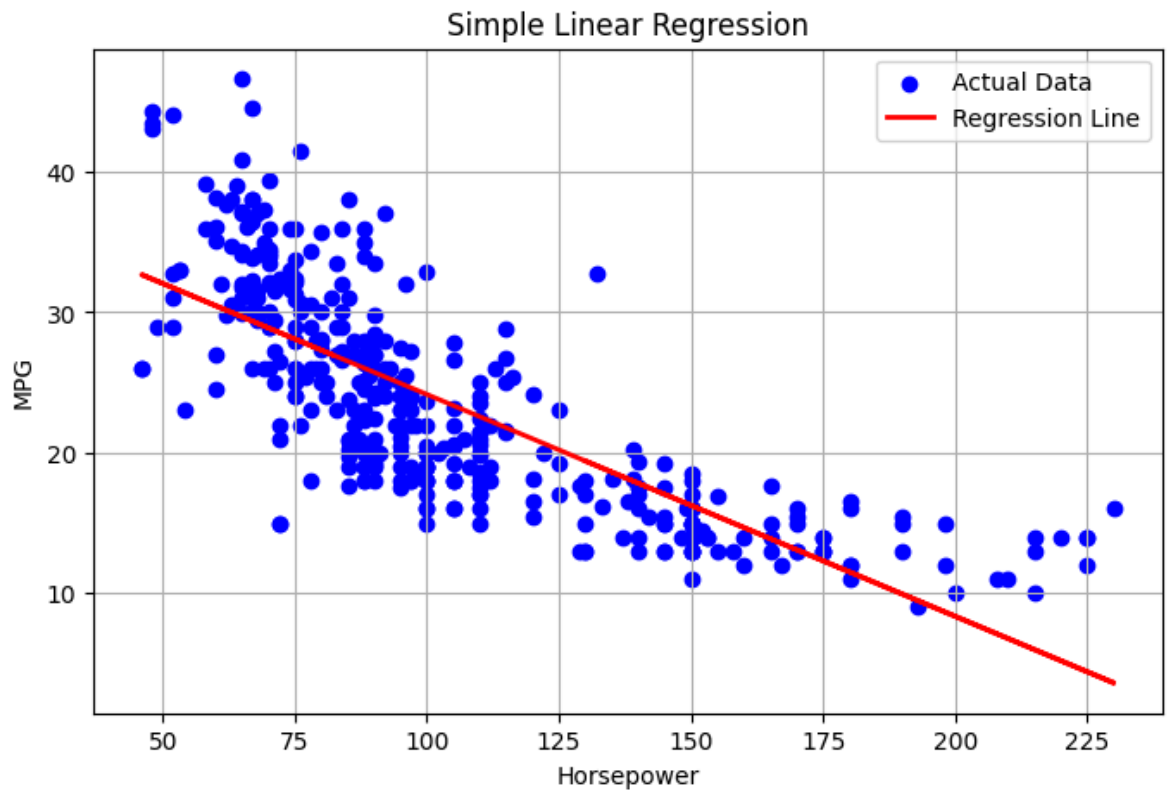
```
plt.ylabel("MPG")
```

```
plt.title("Simple Linear Regression")
```

```
plt.legend()
```

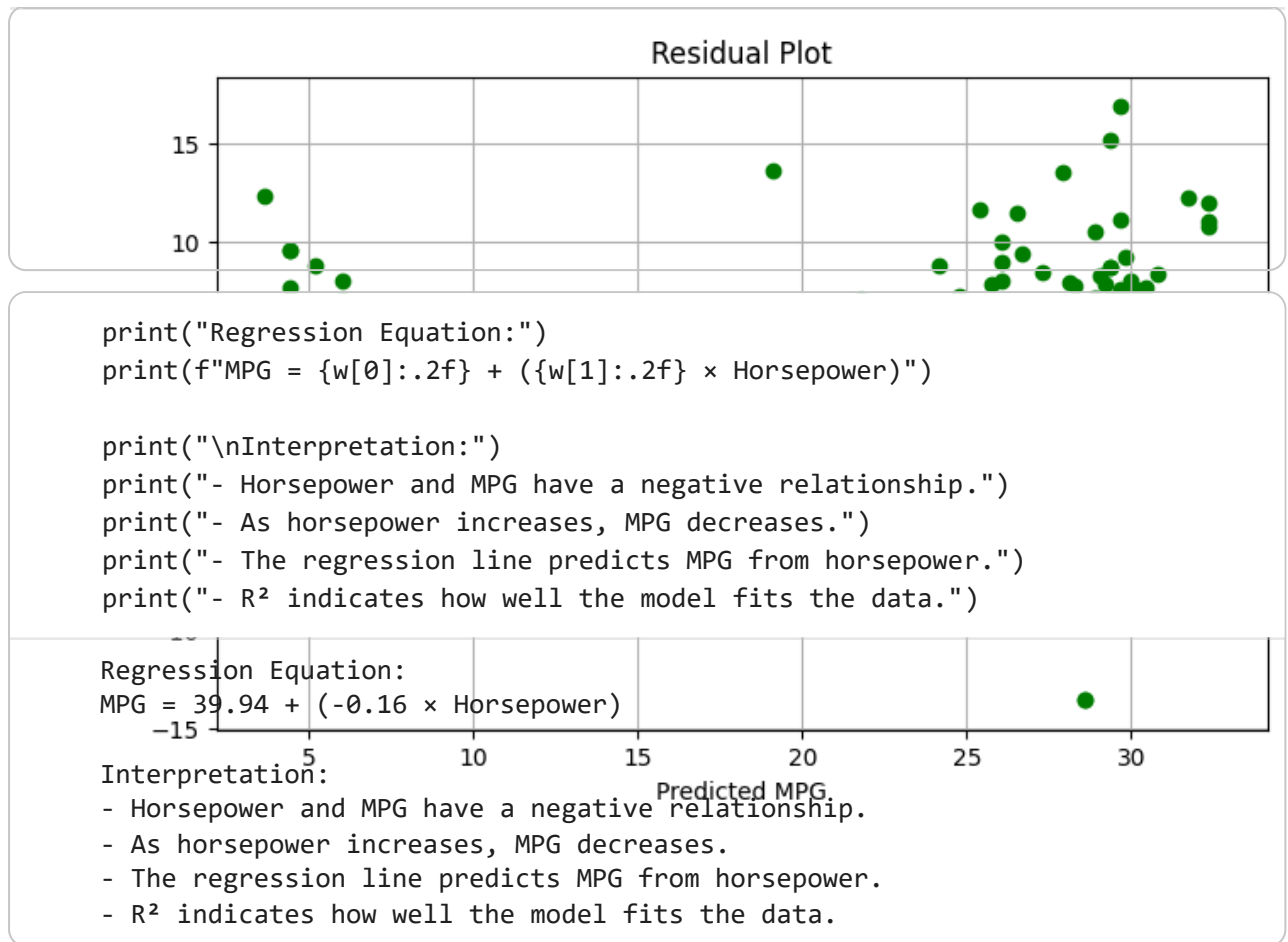
```
plt.grid(True)
```

```
plt.show()
```



```
plt.figure(figsize=(8,5))
plt.scatter(Y_pred, residuals, color="green")
plt.axhline(y=0, color="red", linestyle="--")

plt.xlabel("Predicted MPG")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.grid(True)
plt.show()
```



Conclusion

The experiment successfully implemented Simple Linear Regression using OLS. Horsepower showed a negative relationship with MPG, and evaluation metrics demonstrated reasonable predictive performance.